GUI — Пульт оператора RUS SLAM

> ## ⚠ Legacy (справочно)
>
> Этот документ описывает **справочный** GUI-стек `gui/`. Рабочие экраны робота — в [danilka-revin/Zapret](#) (`slam_gui/`, `run.py`). Протокол в `gui/backend.py` — бинарные кадры 10/16 Б, которых прошивка **не выдаёт**; реальный протокол — текстовый (`SERIAL_PROTOCOL.md`, прошивка fw25).

> **Документ:** `docs/GUI.md` **Объект:** веб-пульт оператора автономного робота-курьера 4WIS/4WID **Каталог исходников:** `gui/` **Версия:** 1.0 · **Дата:** 2026-09 **Связанные документы:** `README.md` · `gui/README.md` · `SOFTWARE_ARCHITECTURE.md` · `SERIAL_PROTOCOL.md` · `ELECTRICAL_AND_PINOUTS.md`

Веб-пульт управления роботом-курьером 4WIS/4WID использует визуальный язык **ZMK Vision**. Отдельно основной экран имеет два клиента: веб-страницу и нативное полноэкранное приложение `gui/native_display.py` на Tkinter/Canvas. Нативный клиент не загружает HTML/CSS/JS, а использует JSON API `backend.py`; его можно упаковать в `.exe`/бинарник через PyInstaller.

---

1. Назначение

GUI — это **пульт оператора**, через который инженер-испытатель управляет автономным роботом-курьером 4WIS/4WID и наблюдает его состояние во время стендовых и полигонных испытаний, а также при подготовке к соревнованиям НТЦ АВТОВАЗ (Задание № 2 «Автономное роботизированное устройство для внутренней логистики»).

Что умеет пульт

- **Телеуправление** — клавиатура (`W/A/S/D/Q/E` + стрелки), виртуальный джойстик (drag мышью), экстренный стоп (Space).
- **Крабовый ход** — переключатель `crab` в верхней панели: при включении все 4 модуля поворачиваются синхронно, кнопки `A/D` дают боковое смещение без поворота корпуса.
- **Автономные сценарии** — автоскан (SLAM-объезд всего помещения), старт маршрута по точкам, программа доставки («А → загрузка → Б → выгрузка → база»), возврат на базу.
- **SLAM-карта** — отрисовка в реальном времени занятой/свободной/ неизвестной области по лидарному скану, путь планировщика A*, путевые точки, площадки А/Б/база, сканирование мира.
- **Симулятор** — если робот не подключён, GUI крутит всю физику и сенсорику сам (лидар, камера, знаки, одометрия, FSM, e-stop). Это сделано для демонстрации, обучения операторов и отладки логики без реального железа.

Чего пульт НЕ делает

- Не подключается к ROS 2 — это **самостоятельный симулятор**. Для интеграции с реальным ROS-стеком используйте `rosbridge_server` + кастомный мост (см. раздел 9).
- Не хранит данные на сервере — все маршруты и группы точек сохраняются в `localStorage` браузера.

---

2. Состав каталога `gui/`

```
gui/
├── main.html        # основной экран робота: моторы, отсек, АКБ (статично на дисплее борта)
├── main.css         # стили основного экрана (тёмная тема, крупные сенсорные элементы)
├── main.js          # веб-визуализация основного экрана (демо-данные → API)
├── native_display.py      # нативный полноэкранный экран (Tk Canvas, без HTML)
├── native_display_core.py # HTTP JSON-клиент для нативного экрана
├── serve.py         # совместимый запуск backend.py: веб-страницы + API
├── index.html       # разметка пульта (sidebar + main + 7 вкладок)
├── styles.css       # стили (ZMK Vision: тёмный сайдбар + светлая зона)
├── console-core.js  # ядро сервисного пульта без DOM: АКБ, протокол, замок, статистика
├── console.js       # окно «Сервис»: PIN-клавиатура, карточки двигателей, АКБ, отчёты
├── app.js           # логика, симулятор, FSM, обработка клавиш, UI
├── vision.js        # рендер карты, камеры, лидара, шасси сверху
├── tests/           # Node/jsdom-тесты веб-пульта и API-клиенты обоих экранов
└── README.md        # краткая инструкция по запуску
```

Файл	Размер	Содержание
`index.html`	~16 КБ	Сайдбар (`brandmark`, навигация, статус связи, профиль) + main (header с кнопкой E-STOP, контейнеры `.view` для 6 вкладок)
`styles.css`	~12 КБ	ZMK Vision: CSS-переменные, тёмная и светлая темы, стили модулей, FSM-индикаторов, журнала
`app.js`	~54 КБ	Глобальный `state`, FSM `["IDLE","READY","NAVIGATE","YIELD_SIGN","WAIT_LIGHT","DELIVER","RETURN","FAULT"]`, планировщик пути A*, физика движения, обработка клавиш, рендер дашборда, программа доставки
`console-core.js`	~35 КБ	Ядро без DOM: `RS.pack` (АКБ 12S3P LiFePO4, OCV-кривая, запас хода, пороги 35,5/33,5 В), `RS.uart` (**текстовые строки протокола fw25 + CRC16/Modbus**), `RS.PinVault` (PIN: SHA-256 + соль, 5 попыток → блокировка 30 с, аудит-цепочка), `RS.StatsStore` (рейсы, энергия, события, CSV/JSON/печать), `RS.fmt`. Работает и в браузере (`window.RS`), и в Node
`console.js`	~28 КБ	Окно «Сервис»: замок ячейки хранения (клавиатура, QR/RFID, смена PIN, аудит), 4 карточки модулей × 2 канала с кнопкой «Ввод» и HEX кадра, панель АКБ с графиками и телеметрией модулей, статистика с KPI и таблицей рейсов, сервисный режим «стенд»
`main.html` · `main.css` · `main.js`	~30 КБ	Веб-основной экран робота: 4 карточки двигателей, кольцо заряда АКБ, PIN-клавиатура с блокировкой после 5 неудач. Подробно — `docs/MAIN_SCREEN.md`
`native_display.py` · `native_display_core.py`	~25 КБ	Нативный основной экран на Tkinter Canvas (без HTML и браузера), HTTP-клиент API `/api/state`, `/api/lock/*`, `/api/audit`; fullscreen, PIN, состояния двигателя и АКБ. Windows/Linux-сборка: `gui/build-native-windows.bat` / `gui/build-native-linux.sh`
`backend.py`	~20 КБ	Сервер борта: статические страницы веб-клиентов (`/` → `main.html`, `/console` → `index.html`), общий JSON API `/api/state`, `/api/lock/*`, `/api/audit`; замок с PIN и аудитом; источники `sim` / `serial` / `ros`. Окон не открывает; флаги `--kiosk` / `--open` запускают браузерный экран
`serve.py`	~1 КБ	Совместимый запуск: `python3 gui/serve.py …` == `python3 gui/backend.py …`
`../deploy/`	файлы автозапуска	`rus-slam-server.service` (сервер), `rus-slam-display.service` + `display.sh` (браузерный экран), `rus-slam-native-display.service` (нативный экран без HTML), `README.md`
`vision.js`	~21 КБ	Рисованная «местность»: `WORLD.walls`, `WORLD.racks`, `WORLD.signs`, `WORLD.stations`, raycast, рендер карты с лидарным сканом, рендер камеры (вид сбоку через raycast), рендер радара, схема шасси сверху

3. Запуск

3.1 Ubuntu 24.04 / 26.04

Демо-режим (без сервера)

```
cd ~/rus_slam/gui
xdg-open index.html      # или просто двойной клик в Файлах
```

Браузер откроет файл по `file://` — пульт сразу запустится.

Полный режим (локальный HTTP-сервер)

```
cd ~/rus_slam/gui
python3 -m http.server 8080
# открыть в браузере: http://localhost:8080
```

Альтернативные серверы

```
# Node.js (если установлен)
npx --yes http-server -p 8080 -c-1

# PHP (если установлен)
php -S 0.0.0.0:8080
```

3.2 Windows 10 / 11

Способ 1 — без сервера (демо-режим)

Двойной клик по `gui\index.html` в Проводнике. Браузер откроет его по `file://` — пульт работает сразу.

Способ 2 — Python (рекомендуется)

1. Установите Python с python.org, обязательно поставьте галочку **Add Python to PATH**.
2. Откройте cmd:

```
cd C:\path\to\rus_slam\gui
python -m http.server 8080
```

3. Откройте в браузере: .

Способ 3 — Node.js

```
cd C:\path\to\rus_slam\gui
npx --yes http-server -p 8080 -c-1
```

4. Архитектура пульта

4.1 Глобальное состояние

Всё состояние хранится в одном объекте `state`, объявленном в `app.js`. Ключевые поля:

Поле	Тип	Назначение
`state.view`	`"dash" | "drive" | "mission" | "sensors" | "safety" | "logs"`	Текущая вкладка
`state.estop`	`bool`	Состояние E-STOP
`state.crab`	`bool`	Режим крабового хода

Поле	Тип	Назначение
`state.vmax`	`number`	Максимальная скорость (м/с), регулируется ползунком
`state.clearance`	`number`	Зазор до препятствий (м) для планировщика
`state.x`, `state.y`, `state.yaw`	`number`	Поза робота в мировой системе (м, рад)
`state.spdVx`, `state.spdVy`, `state.spdWz`	`number`	Реальные (после инерции) скорости
`state.bat`, `state.soc`, `state.current`	`number`	Батарея: В, %, А
`state.waypoints[]`	`[{x,y}]`	Список путевых точек
`state.navPath[]`	`[{x,y}]`	Текущий путь планировщика A*
`state.scan[]`	`[{a, r}]`	Лучи лидара (180 точек на 360°)
`state.modules[]`	`[{id, steer, rpm, temp, homed}]`	4 модуля FL/FR/RL/RR
`state.mission`	`string`	Текущее состояние FSM
`state.program[]`	`string[]`	Очередь шагов программы доставки

4.2 Конечный автомат (FSM)

```
const FSM = ["IDLE", "READY", "NAVIGATE",
             "YIELD_SIGN", "WAIT_LIGHT",
             "DELIVER", "RETURN", "FAULT"];
```

Переходы (в коде `app.js`):

- `IDLE → READY` — оператор дал любую команду движения или нажал «Старт маршрута».
- `READY → NAVIGATE` — авто-режим: строится путь A*, робот едет к следующей waypoint.
- `NAVIGATE → YIELD_SIGN` — лидар/знак «СТОП» или «светофор»: скорость уменьшается, FSM ждёт подтверждения.
- `NAVIGATE → DELIVER` — робот доехал до площадки Б, идёт выгрузка.
- `DELIVER → RETURN` — после `confirmUnload()` едет на базу.
- `* → FAULT` — нажат E-STOP, модуль не отвечает, низкий SOC.
- `FAULT → IDLE` — снят E-STOP.

4.3 Кинематика 4WIS/4WID

Геометрия модулей (из `app.js`):

```
const CHASSIS = [
  { id: "FL", x:  0.34, y:  0.22 },
  { id: "FR", x:  0.34, y: -0.22 },
  { id: "RL", x: -0.34, y:  0.22 },
  { id: "RR", x: -0.34, y: -0.22 },
];
```

Это расстояния от центра масс до точки контакта колеса с полом (м). Функция `compute4WIS(vx, vy, wz)` раскладывает скорость по колёсам и считает требуемые `steer` (рад) и `rpm` для каждого модуля:

```
for (const w of CHASSIS) {
  const vix = vx - wz * w.y;       // скорость точки колеса по X
  const viy = vy + wz * w.x;       // …по Y
  const spd = Math.hypot(vix, viy);
  let steer = Math.atan2(viy, vix);
  // нормализация угла в ±π/2
  if (steer > Math.PI / 2)  steer -= Math.PI;
  if (steer < -Math.PI / 2) steer += Math.PI;
  out.push({ id: w.id, x: w.x, y: w.y, steer, rpm: spd * 180 });
}
```

В **крабовом режиме** (`state.crab === true`) `wz` принудительно обнуляется и `steer` для всех колёс становится одинаковым — робот едет боком или по диагонали без поворота корпуса.

4.4 Главный цикл рендеринга

```
function loop(now) {
  const dt = Math.min(0.05, (now - last) / 1000);
  try {
    applyKeys();      // клавиатура → vx/vy/wz
    publishCmd();     // обновить индикаторы команд
    sim(dt);          // физика: pose, modules, battery, scan
    drawStick();      // виртуальный джойстик
    drawMap();        // SLAM-карта (canvas)
    drawChassisPanel();// шасси сверху (canvas)
    drawCam(now);     // камера перед/зад (canvas, raycast)
    drawLidar();      // радар 360° (canvas)
  } catch (err) { console.error(err); }
  requestAnimationFrame(loop);
}
```

Кадры симуляции идут через `requestAnimationFrame` (~60 FPS), телеметрия обновляется по факту изменения значений.

---

5. Управление

5.1 Клавиатура

Клавиша	Действие
`W` / `↑`	Движение вперёд
`S` / `↓`	Движение назад
`A` / `←`	Поворот влево (или крабовый ход влево при включённом `crab`)
`D` / `→`	Поворот вправо (или крабовый ход вправо)
`Q`	Крабовый сдвиг `vy+`
`E`	Крабовый сдвиг `vy-`
`Space`	E-STOP (мгновенная остановка, обнуляет все скорости, снимает авто-режим)
`H`	Хоминг модулей (отправляет команду 4 модулям «обнулить азимут»)

Реализация в `app.js`:

```
window.addEventListener("keydown", (e) => {
  if (!DRIVE_KEYS.has(e.code)) return;
  e.preventDefault();
  if (e.code === "Space") {
    state.vx = state.vy = state.wz = 0;
    state.keys = {};
    state.auto = false;
```

```
    state.explore = false;
    dragging = false;
    publishCmd();
    return;
  }
  if (e.code === "KeyH") { homeModules(); return; }
  state.keys[e.code] = true;
  applyKeys();
  publishCmd();
});
```

5.2 Виртуальный джойстик

Drag-мышью по холсту `<canvas id="stick">` — координаты пересчитываются в `vx`, `vy`, `wz` с учётом `vmax` и `crab`:

```
function stickFromEvent(ev) {
  const r = stick.getBoundingClientRect();
  const x = (ev.clientX - r.left) / r.width * 2 - 1;
  const y = (ev.clientY - r.top) / r.height * 2 - 1;
  state.vy = clamp(x) * state.vmax * (state.crab ? 1 : 0);
  state.vx = clamp(-y) * state.vmax;
  if (!state.crab) state.wz = clamp(-x) * state.vmax * 1.1;
}
```

5.3 Режимы работы

Режим	Когда включается	Что делает
`TELEOP`	По умолчанию	Оператор управляет клавиатурой/джойстиком
`AUTO · маршрут`	Кнопка «Старт маршрута»	A* по точкам, автопроезд
`AUTO · сканирование`	Кнопка «Автоскан»	Робот ездит по frontier'ам, пока карта не покроется >62 %
`AUTO · программа`	Кнопка «Запустить мою программу»	Шаги из `state.program` (А→загрузка→Б→выгрузка→база)
`AUTO · на базу`	Кнопка «На базу» / SOC < 18 %	Доехать до dock-станции, начать зарядку
`E-STOP`	Кнопка E-STOP / Space	Гейт закрыт, скорости обнулены

5.4 Программа доставки (список шагов)

Кнопки «Добавить шаг» добавляют в `state.program` записи:

Кнопка	Запись	Описание
🚗 ехать на А	`goA`	Подъехать к площадке загрузки
🚗 ехать на Б	`goB`	Подъехать к площадке выгрузки
📦 взять груз	`load`	Ожидание подтверждения загрузки
📦 сдать груз	`unload`	Ожидание подтверждения выгрузки
📍 по точкам	`goP`	Проехать по кликнутым точкам на карте
⏸ пауза 2 с	`wait`	Стоять 2 секунды
🏠 на базу	`dock`	Возврат на док-станцию
🔋 зарядиться	`charge`	Зарядка до `chargeTo` %
🔁 цикл	`loop`	Повтор с шага 1

После составления очереди нажмите «Запустить мою программу» — FSM выполнит шаги сверху вниз, в нужных местах потребует ручного подтверждения «груз взят» / «груз сдан».

---

6. SLAM и планирование пути

6.1 Лидар

180 лучей, развёрнутых на 360°, дальность 10 м. Каждый кадр:

```
for (let i = 0; i < LIDAR_BEAMS; i++) {
  const a = state.yaw + (i / LIDAR_BEAMS) * Math.PI * 2;
  state.scan.push({ a, r: raycast(state.x, state.y, a, LIDAR_MAX) });
}
```

`raycast` бросает луч по сетке `WORLD.walls` и `WORLD.racks` и возвращает расстояние до первого пересечения.

6.2 Occupancy grid

Сетка 320 × 240 ячеек, разрешение 0.12 м/яч, мировые координаты от -18 до +18 м по X и от -12 до +12 м по Y. На каждом кадре `integrateScan()` обновляет значения:

Код	Цвет в vision.js	Значение
`UNK = 0`	#151c19 (фон)	Неисследованная область
`FREE = 1`	#3c433c	Свободный проезд
`OCC = 2`	#8a6a48	Стена / стеллаж / препятствие

Также строится `distGrid` — расстояние в метрах от каждой свободной ячейки до ближайшего препятствия (двупроходный Chamfer-distance, пересчитывается лениво при изменении карты).

6.3 Планировщик A*

Классический A* по 8-связной сетке с эвристикой Евклида и учётом штрафа за близость к стенам (`wallCost`):

```
function planPath(gx, gy) {
  /* 1. проверка целевой точки
     2. обычная очередь с приоритетом по f = g + h
     3. для каждого соседа: шаг = 1 (прямой) или 1.41 (диагональ)
        умножить на wallCost(nx, ny)
     4. восстановление пути через массив came[]                    */
}
```

Параметр `clearance` (ползунок в UI, 0.5–1.5 м) задаёт **минимальный зазор** до препятствий: если `wallCost` ячейки выше порога, ячейка считается непроходимой (используется инфляция препятствий `inflatedOcc`).

6.4 Карта и взаимодействие

- ЛКМ на карте — добавить точку маршрута (`state.waypoints.push`).
- ПКМ — убрать последнюю точку.
- Колесо мыши — зум (масштаб 8…50 px/м).
- Drag ЛКМ — панорамирование.
- Кнопка «К роботу» — центрировать камеру на текущей позе.
- Кнопка «Снимок карты» — PNG текущего вида (`canvas.toDataURL`).

---

7. Вкладки (view)

Sidebar содержит 7 кнопок, переключающих `state.view`:

Вкладка	Что показывает
Пульт	Основной экран: метрики (АКБ/Путь/Скорость/Гейт), SLAM-карта, панель управления, программа доставки
Ходовая	Карточки модулей 4WIS/4WID + схема шасси сверху с углами поворота колёс и стрелкой «груз взят»
Миссия	FSM-индикатор (IDLE / READY / NAVIGATE / YIELD_SIGN / WAIT_LIGHT / DELIVER / RETURN / FAULT), журнал событий, метрики программы
Сенсоры	Радар 360° на canvas, лог детекций знаков, статус лидара
Безопасность	Состояние E-STOP, свежесть `@F` и подтверждения `brake_id`, CRC-ошибки, ток АКБ, состояние гейта `/cmd_vel`
Журнал	Полный лог событий (200 последних записей), фильтр по уровням `ok/warn/err`
Сервис	Ячейка хранения (PIN/QR/RFID, аудит), двигатели (поле ввода + «Ввод», строка протокола), АКБ 12S3P, статистика и отчёты — см. `docs/SERVICE_CONSOLE.md`

7.1. Вкладка «Сервис» (окно обслуживания)

Технический план и полное описание — `docs/SERVICE_CONSOLE.md`. Кратко:

1. **Ячейка хранения** — заводской PIN 2580, клавиатура 1…0/⌫/C, кнопки QR и RFID, смена PIN и сброс, журнал доступа с хеш-цепочкой (подделка записи обнаруживается). Открытие ячейки снимает `state.cargoLock` и блокирует старт миссии до закрытия.
2. **Двигатели** — 4 карточки (FL #1, FR #2, RL #3, RR #4): рулевой канал (угол, °) и тяговый (тяга, %), у каждого поле ввода + кнопка **«Ввод»**, джоги ±5°/±45° и −20/0/+20 %, хоминг. Каждый «Ввод» формирует реальную **текстовую** строку протокола fw25 (например `w FL 20.0`, `v 40 f`, `b`) и показывает её в журнале; HEX-кадров в протоколе робота нет (§ `docs/SERIAL_PROTOCOL.md`). Интерлоки: команды разрешены только в режиме «стенд» и вне E-STOP.
3. **АКБ** — 12S3P LiFePO4 (30…43,8 В, 714,2 Вт·ч): SOC по OCV, ток, мощность, разброс ячеек, запас хода, телеметрия `@F`/`@TLM` по модулям, графики SOC и скорости.
4. **Статистика** — KPI (рейсы, пробег, энергия, расход, груз, объезды, знаки, светофор, замок, сбои), таблица рейсов, экспорт CSV/JSON и печатный отчёт.

Сервисный режим «стенд» (тумблер в панели двигателей) обездвиживает корпус, сбрасывает авторежим и телеуправление и разрешает ручную подачу команд модулям.

Улучшения интерфейса окна (И-1…И-10): липкая панель быстрых действий с живыми статусами, палитра команд `Ctrl+K` и горячие клавиши (`Alt+1…7`, `Alt+L/S/X/Z/A`), монитор смены (кольцо SOC, рейсы, светофор, последний знак, заголовок вкладки), валидация полей с автоклампом, «мёртвая рука» на кнопках тяги (±20 % только при удержании), звуковые сигналы Web Audio, собственные модальные окна вместо `confirm/alert`, журнал сервисных действий с фильтрами и копированием кадра, режим «крупный интерфейс» и доступность (`aria-live`, `focus-visible`), синхронизация двух окон пульта через `BroadcastChannel/storage`. Подробная таблица — `docs/SERVICE_CONSOLE.md` §8.1.

Проверка окна:

```
node gui/tests/console.test.js                    # ядро, без зависимостей
NODE_PATH=/tmp/harness/node_modules node gui/tests/jsdom.console.test.js   # интеграция (нужен jsdom)
```

8. Тема, маршруты, e-stop

8.1 Тема

Две темы — тёмная (по умолчанию, сайдбар `#101b17`, фон `#0b120f`) и светлая (фон `#f3f5f7`). Выбор сохраняется в `localStorage`:

```
const KEY = "rus_slam_theme";
applyTheme(localStorage.getItem(KEY) !== "light");
```

Переключение — кнопка «Тёмная/Светлая» в шапке.

8.2 Сохранённые группы точек

Пользовательские маршруты (`state.waypoints`) можно сохранять в `localStorage` под именем. Структура хранения:

```
const ROUTES_KEY = "rus_slam_route_groups";
// [{ name: "цех-А", points: [{x, y}, ...] }, ...]
```

Операции: «Сохранить группу», «Загрузить», «Удалить». Список доступных групп подгружается в `<select id="route-list">`.

8.3 E-STOP

Кнопка `<button id="btn-estop">` в шапке переключает `state.estop`. При включении:

- `state.vx = state.vy = state.wz = 0` (мгновенно).
- `state.auto = state.explore = false` (все авто-режимы прерваны).
- FSM → `FAULT`.
- Индикатор «Связь с базой» пульсирует красным.
- Кнопка подсвечивается красным, в журнал пишется `E-STOP asserted` уровня `err`.

Повторное нажатие снимает E-STOP и возвращает FSM в `IDLE`.

---

9. Интеграция с ROS 2 (опционально)

Текущий пульт — **самостоятельный симулятор** и не публикует данные в ROS. Чтобы связать его с реальным роботом, нужны два слоя:

1. `rosbridge_server` на бортовом компьютере Ubuntu:

```
   sudo apt install -y ros-$ROS_DISTRO-rosbridge-server
   ros2 launch rosbridge_server rosbridge_websocket_launch.xml
```

2. **JS-мост** в `gui/app.js`, который публикует команды в `/cmd_vel_nav` и подписывается на `/odom`, `/battery_state`, `/module_status`, `/map` (см. `docs/SOFTWARE_ARCHITECTURE.md`).

Ниже — каркас, который можно вставить в `app.js` после объявления `state`:

```
// 1. Конфиг подключения (можно хранить в localStorage)
const ROS_URL = localStorage.getItem("ros_url") || "ws://localhost:9090";

// 2. Подключение
let ros = null;
function connectRos() {
  ros = new WebSocket(ROS_URL);
  ros.onopen = () => {
    subscribe("/odom", "nav_msgs/msg/Odometry", (msg) => {
      state.x = msg.pose.pose.position.x;
      state.y = msg.pose.pose.position.y;
      const q = msg.pose.pose.orientation;
      state.yaw = Math.atan2(
        2 * (q.w * q.z + q.x * q.y),
        1 - 2 * (q.y * q.y + q.z * q.z)
      );
    });
    subscribe("/battery_state", "sensor_msgs/msg/BatteryState", (msg) => {
      state.bat = msg.voltage;
      state.soc = msg.percentage * 100;
    });
    subscribe("/module_status", "rus_slam_interfaces/msg/ModuleStatus", (msg) => {
      const m = state.modules.find(x => x.id === msg.id);
      if (m) { m.steer = msg.steer_angle * 180 / Math.PI;
```

```
                m.rpm  = msg.drive_rpm;
                m.temp = msg.module_temperature;
                m.homed = msg.homed; }
    });
  };
  ros.onclose = () => setTimeout(connectRos, 2000);
}
function subscribe(topic, type, cb) {
  ros.send(JSON.stringify({ op: "subscribe", topic, type }));
  ros._subs = ros._subs || {};
  ros._subs[topic] = cb;
}
ros.addEventListener("message", (e) => {
  const m = JSON.parse(e.data);
  if (m.op === "publish" && ros._subs && ros._subs[m.topic]) {
    ros._subs[m.topic](m.msg);
  }
});

// 3. Публикация команд (в функции sim или в applyKeys)
function publishCmd() {
  if (!ros || ros.readyState !== 1) return;
  ros.send(JSON.stringify({
    op: "publish",
    topic: "/cmd_vel_nav",
    type: "geometry_msgs/msg/Twist",
    msg: { linear: { x: state.vx, y: state.vy, z: 0 },
           angular: { x: 0, y: 0, z: state.wz } }
  }));
}
```

Отключение симулятора при наличии ROS-моста: в начало функции `sim(dt)` добавьте `if (ros && ros.readyState === 1) return;` — тогда `state.x/y/yaw` будут приходить только из `/odom`.

> ⚠ Эти правки **не входят** в исходный пульт автора. Они описаны как готовая «точка включения» — добавляйте их сами под свои требования к реальному стеку.

---

10. Дизайн (ZMK Vision)

Токен	HEX	Где используется
Sidebar bg	`#101b17`	Фон боковой панели
Accent	`#d5ff45`	Активные элементы, точка маршрута, лучшая точка, индикатор «заряд готов»
Light surface	`#f3f5f7`	Фон светлой рабочей зоны
Dark canvas bg	`#0b120f`	Фон камеры (вид сбоку)
Warning red	`#e4503e`	Площадка «Б», кнопка E-STOP, лидар-препятствия
OK green	`#7ddc52`	Площадка «А», пройденные точки, пройденный путь
Light blue	`#7ec8ff`	Текущий спланированный путь A*

Шрифты: **Inter** (текст, моноширинный) + **Manrope** (заголовки Sidebar). Оба подгружаются с Google Fonts прямо в <head>.

---

11. Совместимость и ограничения

Браузер	Поддержка
Chrome / Edge 110+	✅ полная
Firefox 110+	✅ полная
Safari 16+	✅ полная (включая iPad)
Браузеры без Pointer Events	⚠ drag-джойстик не работает, клавиатура — да

- Зависимость только от <canvas> 2D, `requestAnimationFrame`, `localStorage`, `WebSocket` (опционально, для ROS-моста).
- Никаких npm-пакетов, никаких сборщиков — просто открыть.
- Загрузка CSS и JS в одном файле каждый, минификация не требуется (размеры приемлемые).

---

12. Известные особенности

- **Несколько вкладок одного пульта** в браузере делят `localStorage` — сохранённые маршруты синхронизируются через событие `storage`.
- **Глобальный** `state` — нет изоляции между экземплярами пульта (открытие двух окон приведёт к двум независимым симуляторам, но конфликтующим записям в `localStorage`).
- **Симулятор не считает** инерцию колёс, проскальзывание, трение — это идеализированная физика для демонстрации логики, не для валидации кинематики.
- **Карта сетки** фиксированного размера 320 × 240 и не сдвигается — при выезде за её границы (например, `|x| > 15.6`) включается «возврат в центр».

---

13. Диагностика

Симптом	Причина	Решение
Карта пустая, робот не движется	Другой экземпляр пульта в той же вкладке	Перезагрузить страницу (`Ctrl+F5`)
Программа доставки не запускается	FSM в `FAULT`	Снять E-STOP, проверить модули
Колесо мыши не зумит карту	`file://` не передаёт wheel-events в preventDefault	Запустить через `python3 -m http.server`
Шрифты Inter/Manrope не подгружаются	Нет интернета	Не критично, fallback на system-ui
Сайдбар «съедает» контент	Узкое окно (< 1100 px)	Скрыть панель, развернуть окно

---

14. Лицензия и авторы

- **Лицензия:** MIT (как и весь проект RusSLAM).
- **Автор GUI:** Данил Ревин · команда RusSLAM · НТЦ АО «АВТОВАЗ».
- **Визуальный язык:** ZMK Vision.
- **Контакт:** см. `README.md` в корне репозитория.